@ 7th October - Wednesday :- & 11th October - Sunday :-

Title : Basics of Python :-

* Data : It's a collection of information.
* (Numbers, String, List, Set, Tuple, dictionary)
* Data types : int, float, complex, string, Boolean.
* Numeric datatypes : int, float, complex.

Eg :- int : my_age = 15
float : value_of_pi = 3.142
complex : x = -5j
y = 2 + 4j
z = 22j

* Boolean = stores the value as either True or False
* String = Can store alphabets, numbers & special characters. A string is always enclosed in double quotes "" or single quotes ''

Eg :- "Python" "CH62H16" "Harry@123.com" "1234$"

* In python Variables are used to ~~store~~ hold the different ~~data~~ types of data & the value of the variable can be changed at any time Based on the need.

↓
mainly used to store the data.

Data types define the type of data & variables are used to ~~the~~ store the different types of data. Variable = ~~the~~ School Bag to store different things Based on requirement & need.

* Variable Declaration : Each variable is given by a unique name using which we store, access, & modify the values.

-> Such a name can contain alphanumeric characters, underscore but no space or special characters allowed. The 1st letter should be Alphanumerical Character or Underscore.

-> Variables can store different types of values:

Eg: class_strength = 95 # int
_subject = "Maths" # String
Mark1 = 75.5 # Float

* Input statement: used to take input.

Name = input("Please enter your name:")

* Print statement: Used to display the output.

print = ("Name of the student is:", Name)

(How to use & operate on Variables).

* Operators: An operator is a symbol that helps perform specific mathematical, relational or logical operation. (+ - % * ÷ //)

Eg: A = 1 + 2
↓ → operators
→ values

Variable that holds the results.

* Types of operators : (i) Arithmetic operators.
(ii) Assignment operators.
(iii) Relational operators.
(iv) Logical operators.

(i) Arithmetic operators :

-> Addition (+)
-> Subtraction (-)
-> Multiplication (x)
-> Division (/)
} Basic ones.
-> Integer Division (//)
-> Modulo (%)
} In addition in python

Eg: print (3+5) # output : 8
print (42*2) # output : 84
print (5/2) # output : 2.5
print (5//2) # output : 2
print (5%2) # output : 1

-> Both (/ and //) are division operators but double division (//) operator does integer division so it omits the decimal point.

-> % (modulo operator) gives the remainder of the division as a result.

(ii) Assignment operators: as it's name says is used for assigning a value to a variable.

-> operators are : =, +=, -=, *=, /=, %=

-> Simple assignment (=)
-> Short hand assignment addition (+=)
-> Short hand assignment subtraction (-=)
-> Short hand assignment multiplication (*=)
-> Short hand assignment division (/=)
-> Short hand assignment Modulo (%=)

Eg: ① marks = 25 print(marks) # output = 25
marks += 3 print(marks) # output: 28 add RHS to LHS

(iii) Relational Operators: are ~~used~~ comparison operators used to compare two values. It returns the Boolean value i.e., true or false.

-> Greater than (>)
-> Lesser than (<)
-> Greater than or equal to (>=)
-> Lesser than or equal to (<=)
-> Equals to (==)
-> Not equals to (!=)

Eg: a = 7 b = 5
print (a > b) # True
print (a < b) # False
print (a >= b) # True
print (a <= b) # False
print (a == b) # False
print (a != b) # True

(iv) Logical operators: we mainly have 3 logical operators (AND, OR, NOT)

Eg: A B A and B (multiplication operator)

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

A	B	A or B (Additional operator)
True	True	True
True	False	True
False	True	True
False	False	False

A	not A (reverse value)
True	False
False	True

Selection construct.

Programming constructs Part-1 : Selection control structure.

* **Selection control structure** : allows one set of statements to be executed if a condition is true & another set of statements be executed if condition is false. (Based on some conditions, control flows to different set of statements).

* **types** : → if
→ if-else
→ multiple if's.

} control statements.

① **if statement syntax** -

condition to be checked

if ← if (condition) :→ colon makes the start of if block of statements.

keyword in python

statement 1
---
statement n

} statements to be executed when condition is true

↔

observe the tab space given for if-block statements (indentation)

Eg:-

```
time = "evening"
if (time == "evening"):
    print ("go to home")
```

output

go to home

(ii) If - else statement :-

Syntax:

```
if (condition):
    statement 1
    ---
    statement n
else-keyword -> else:
in python       statement 1
                --
                statement n
```

e.g:-

```
student_name = "Arya"
marks = 75
if (marks >= 75):
    print (student_name, "your marks is", marks, "you are in A sec")
else:
    print (student_name, "marks is", marks, "you are in B sec")
```

output:

Arya your marks is 75 you are in A sec

(iii) multiple if's :- Syntax :-

```
if (condition):
    statement 1
elif (condition):
    statement 1
else:
    statement 1
```

Eg: time = 4

```
if (time = 4):          | (Time >= 1 and time <= 4):
    print ("it's evening"):
elif (time = 6):            (time >= 5 and time <= 8):
    print ("it's morning")
else:
    print ("it's night")
```

Output:

it's evening

☆ Iteration control structures:

* **Iteration:** For doing repeated tasks. (certain statements will be executed repeatedly).
  - → For
  - → While

  } also called looping statements.

① while:

condition need to be check
↓
while (condition) :→ colon symbol marks the start of the while block of statements.
    statement 1
    - - statement n } to be executed when the condition is true.

Eg:

```
pen_count = 10
pen_taken = 0
while (pen_taken > 9):
    print("All pens are soled", pen_taken)
```

output:
All pens are soled 10

(1) For syntax:- user defined variable name.

in is a keyword in Python.

A sequence of values

```
for number in 1,2,3,4,5:
    print ("The current number is", number)
```

↓ statements to be repeatedly executed as long as there are values in the sequence.

Eg:-

```
extra-classes = ['Vocal', 'Dance', 'Singing', 'Instrumental']
print ("These are all your classes today")
for subjects in extra-classes:
    print (subjects)
```

output:

These are all your classes today
Vocal
Dance
Singing
Instrumental

Loop control statements: (to control loops)

-> There are the 3 loop controls in python:

(i) Break
(ii) continue
(iii) Pass.

(i) Break:

```
result = True
for marks in 65, 45, 89, 16, 69:
    if marks < 35:
        result = False
        break
    print (marks)
if (not(result)):
    print ("fail")
```

output:

```
65
45
89
fail
```

(ii) Continue:

```
sum = 0
for mark in 65, 45, 89, 16, 69:
    if mark < 35:
        continue
    sum += mark
    print (mark)
print ("Total marks excluding fail sub :", sum)
```

output: 65
45
89
69
- - - - - - - - : 268

(iii) Pass:

```
reward_count = 0
for marks in 65, 98, 59, 43, 68:
    if marks < 90:
        pass
    else:
        reward_count += 1
print ("You have earned", reward_count)
```

output: You have earned 1

* Collections: are used to group the data/information & store them together.

* different types of collections in python are:-

(i) List
(ii) String
(iii) Set
(iv) Tuple
(v) Dictionary.

(i) List: are the group of elements that can be stored in a specific order & whose values can change.

List representation:

Attendancelist = [1, 2, 3, 4, 5]

declaring list with name = Attendancelist

list is represented using square brackets. separated by comma.

* Index: the representation of value's place starts from 0 (zero) & continues to n-1.

* How to access different information from the list in python:-

len(Attendancelist) = syntax to know length of a list.

Attendancelist[0] = To find / see / display the first element with index 0 zero in the list.

attendancelist[4] = to know 5th element in the list of 5 elements as index starts from 0 & nth value index = n-1 //

attendancelist.append(6) = to add the new element to the list at the end.

* Adding 2 lists:

```
list1 = [1, 2, 3, 4]
list2 = [5, 6, 7, 8]
list3 = list1 + list2
print(list3)                    #output [1, 2, 3, 4, 5, 6, 7, 8]
```

* List slicing: the concept comes into the frame when you want to access some specific elements in the list, not the entirelist.

```
marks = [78, 88, 90, 97, 67]
marks[2:5]                      #output [90, 97, 67]
```

↑ to get the elements from the 2nd to the 4th element (position 2 to 5-1)

marks[3:] -> to print from 3rd element to the last element. [97, 67] -> output

(11) String representation: Each value in a string is called character. we can access the position of the characters in the string's based on their index positions. name = "Roshan"

* Accessing the 5th alphabet in the name "Roshan":

```
name[4]          #output 'a'
```

* To count the number of occurrences of alphabet 'o' in the name:

```
name.count('o')     #output 1
```

* To convert all the alphabets in your name in upper case:

```
name.upper()        #output 'ROSHAN'
```

* To convert all the alphabets in your name to lower case:

```
name.lower()        #output 'roshan'
```

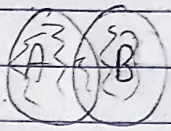
(iii) Sets: they are unordered collection of items which contain only unique values & they are represented using curly braces {}.

* Set representation: rollno = {1, 2, 3, 4, 5, 6, 7, 8}

2 more sets music = {1, 2, 3, 4, 5}
Karate = {5, 6, 7, 8}

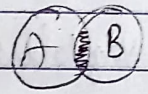
* operations on sets: (union, intersection, & symmetric difference) = mathematical operations

union: music | Karate



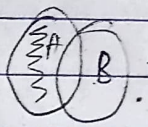
{1, 2, 3, 4, 5, 6, 7, 8}

intersection: music & karate



{5}

set-difference: music - Karate



{1, 2, 3, 4}

* To add a new element to set (music):-

music.add(9) # output {1, 2, 3, 4, 5, 9}

* To remove a element from set (music):-

music.remove(1) # output {2, 3, 4, 5, 9}

(IV) Tuples: immutable lists. when we want to store a element whose values can't be changed then we use a collection called tuple.

☆ Tuple representation: we use rounded braces. ()

* Creating a tuple:

Vegetables = ('spinach', 'chilli', 'potato')

* Accessing the 3rd element = index = 3:-

Vegetables[2] # output 'potato'

* To combine 2 tuples:

combo = vegetables + vegetables

output ('spinach', 'chilli', 'potato', 'spinach', 'chilli', 'potato')

(v) Dictionary: In python, there is a concept of key-value pairs & such a collection is called as a dictionary.

* creating a dictionary: Corresponding marks to a name.

```
Student_marks = {'raman': 20, 'yogesh': 95, 'Rahul': 30}
print (Student_marks)
#output {'raman': 20, 'yogesh': 95', 'rahul': 30}
```

name of the student is the key & marks is the value.

* Accessing any value using key:

```
print (Student_marks['raman'])

#output: 20
```

* Adding or updating an element in the dictionary:-

```
Student_marks.update ({'brenda': 98})

print (Student_marks)
#output {'raman': 20, 'yogesh': 95, 'rahul': 98}
```